

## Write a Python Program to Implement Alpha-Beta Pruning Algorithm for Gaming

### Aim

To develop a Python program that implements the Alpha-Beta Pruning Algorithm for game tree search and finds the optimal move by reducing unnecessary node evaluations.

### Objective

- To understand Alpha-Beta Pruning in Artificial Intelligence.
- To optimize the Minimax Algorithm.
- To reduce the number of nodes evaluated in a game tree.
- To improve the efficiency of game-playing algorithms.

### Theory

Alpha-Beta Pruning is an optimization technique for the **Minimax Algorithm**. It eliminates branches of the game tree that do not affect the final decision. Two values are maintained during the search:

- **Alpha ( $\alpha$ ):** Best value that the maximizing player can guarantee.
- **Beta ( $\beta$ ):** Best value that the minimizing player can guarantee.

Whenever **Alpha becomes greater than or equal to Beta ( $\alpha \geq \beta$ )**, the remaining branches are pruned because they cannot influence the final result. Thus, Alpha-Beta Pruning produces the same optimal result as Minimax while evaluating fewer nodes.

### Algorithm

**Step 1:** Define the game tree leaf node values.

**Step 2:** Initialize Alpha =  $-\infty$  and Beta =  $+\infty$ .

**Step 3:** Start from the root node.

**Step 4:** If it is a maximizing node, update Alpha.

**Step 5:** If it is a minimizing node, update Beta.

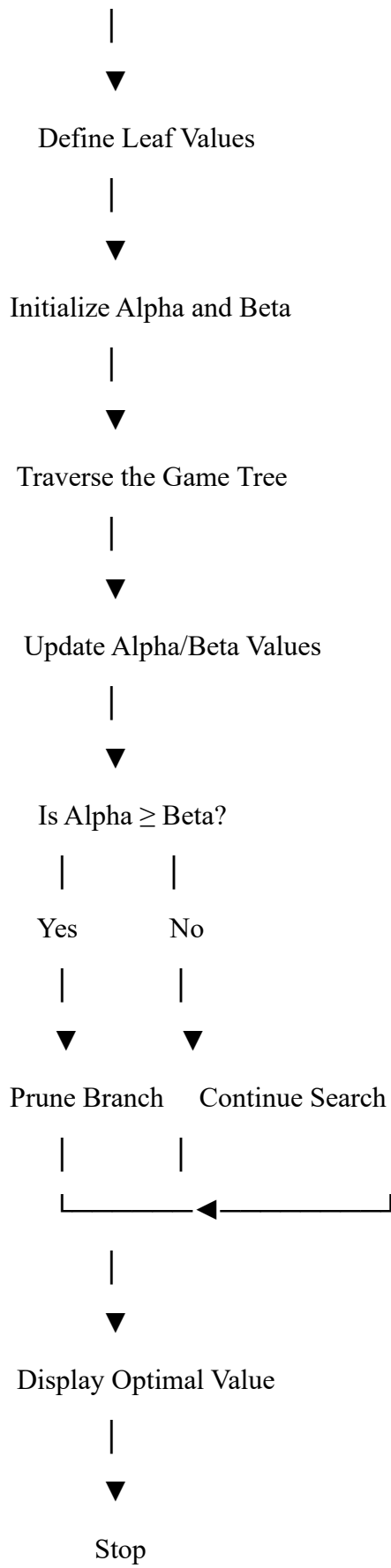
**Step 6:** Whenever Alpha  $\geq$  Beta, stop exploring that branch (Prune).

**Step 7:** Continue until all necessary nodes are evaluated.

**Step 8:** Display the optimal value.

### Flowchart

Start



## Python Program

```
import math
```

```
def alphabeta(depth, node, maximizing, values, alpha, beta, height):
```

if depth == height:

```
return values[node]
```

if maximizing:

```
best = -math.inf
```

```
for i in range(2):
```

```
value = alphabeta(depth + 1, node * 2 + i,  
                  False, values, alpha, beta, height)
```

```
best = max(best, value)
```

```
alpha = max(alpha, best)
```

```
if beta <= alpha:
```

break

return best

else:

```
best = math.inf
```

```
for i in range(2):
```

```
value = alphabeta(depth + 1, node * 2 + i,  
                  True, values, alpha, beta, height)
```

```

        best = min(best, value)

        beta = min(beta, best)

    if beta <= alpha:
        break

    return best

values = [3,5,6,9,1,2,0,-1]

height = 3

print("Optimal Value:",
      alphabeta(0,0,True,values,-math.inf,math.inf,height))

```

### Sample Output

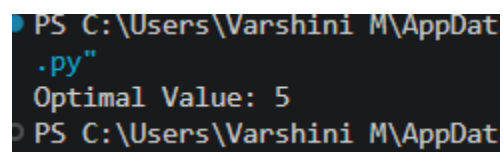
Optimal Value: 5

### Result

The Python program successfully implemented the Alpha-Beta Pruning Algorithm and found the optimal game value by pruning unnecessary branches.

### Conclusion

Alpha-Beta Pruning significantly improves the performance of the Minimax Algorithm by avoiding unnecessary node evaluations while still producing the optimal solution.



```

PS C:\Users\Varshini M\AppData
.py"
Optimal Value: 5
PS C:\Users\Varshini M\AppData

```

```

C:\Users\Varshini M > OneDrive > Desktop > Here!!! > ARTIFICIAL INTELLIGENCE > VSC EXPs > EXP1.py > alpha
1  import math
2
3  def alphabeta(depth, node, maximizing, values, alpha, beta, height):
4
5      if depth == height:
6          return values[node]
7
8      if maximizing:
9          best = -math.inf
10
11         for i in range(2):
12             value = alphabeta(depth + 1, node * 2 + i,
13                               False, values, alpha, beta, height)
14
15             best = max(best, value)
16             alpha = max(alpha, best)
17
18             if beta <= alpha:
19                 break
20
21         return best
22
23     else:
24         best = math.inf
25
26         for i in range(2):
27             value = alphabeta(depth + 1, node * 2 + i,
28                               True, values, alpha, beta, height)
29
30             best = min(best, value)
31             beta = min(beta, best)
32
33             if beta <= alpha:
34                 break
35
36         return best
37
38 values = [3,5,6,9,1,2,0,-1]
39
40 height = 3
41
42 print("Optimal Value:",
43       alphabeta(0,0,True,values,-math.inf,math.inf,height))
44

```